IT3030 – Programming Application Frameworks
BSc (Hons) in Information Technology
Faculty of Computing
SLIIT

2026 - Tutorial

React JS – State Management

Sample Codes

```
import { useState } from "react";

export default function Sample() {
  const [count, setCount] = useState(0);
  const [amount, setAmount] = useState(0);

  const hadleIncrement = () => {
    setCount(count + 1);
  };
  const handleDecrement = () => {
    setCount(count - 1);
  };

  const delayCount = () => {
    setTimeout(() => {
      setCount(count + 1);
    }, 2000);
  };

  const handleOnchange = (event) => {
    setAmount(event.target.value);
  };

  const handleIncrementbyAmount = () => {
    setCount(count + parseInt(amount, 10));
  };

  return (
    <div>
      <h1>Hello, PAF!</h1>
      <br />
      <p>Count = {count}</p>
      <br />
      <button onClick={hadleIncrement}>Increment</button>
      <button onClick={handleDecrement}>Decrement</button>

      <br />
      <button onClick={delayCount}>Delay Count</button>
      <br />
```

```
    <label htmlFor="amount">Input Number</label>
    <input
      type="number"
      name="amount"
      onChange={handleOnchange}
      value={amount}
    />
    <button onClick={handleIncrementbyAmount}>Increment by Amount</button>
  </div>
 );
}
```

Async with Fetch API

```
import { useState, useEffect } from "react";

export default function SampleAsync() {
  const [data, setData] = useState(null);

  useEffect(() => {
    setTimeout(() => {
      const sampleData = {
        name: "John Doe",
        age: 30,
        email: "john@example.com",
      };
      setData(sampleData);
    }, 2000); // Simulating a 2-second delay
  }, []);

  return (
    <div>
      <h1>User Info</h1>
      {data ? (
        <div>
          <p>Name: {data.name}</p>
          <p>Age: {data.age}</p>
          <p>Email: {data.email}</p>
```

```
        </div>
      ) : (
        <p>Loading data...</p>
      )}
    </div>
  );
}
```

```
import { useState, useEffect } from "react";

export default function SampleAsync() {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchData = async () => {
      try {
        const response = await fetch(
          "https://jsonplaceholder.typicode.com/users/1"
        );
        if (!response.ok) {
          throw new Error("Network response was not ok");
        }
        const userData = await response.json();
        setData(userData);
      } catch (error) {
        console.error("Error fetching data:", error);
      } finally {
        setLoading(false);
      }
    };

    fetchData();
  }, []);
```

```
  return (
    <div>
      <h1>User Info</h1>
      {data ? (
        <div>
          <p>Name: {data.name}</p>
          <p>Age: {data.age}</p>
          <p>Email: {data.email}</p>
        </div>
      ) : (
        <p>Loading data...</p>
      )}
    </div>
  );
}
```

Context API

1. Create a new vite project

npm create vite@latest

2. Install dependencies

npm install

3. Run the project
4. Remove unnecessary codes.
5. Below code is for simple app with state management

```
function App() {
  const [count, setCount] = useState(0);
  const handleClick = () => {
    setCount(count + 1);
  };

  return (
    <div>
```

```
    <h1>Count: {count}</h1>
    <button onClick={handleClick}>Increment</button>
  </div>
 );
}
```

6. Extending the apps functionality by introducing a child component

```
function App() {
 const [count, setCount] = useState(0);
 const handleClick = () => {
  setCount(count + 1);
 };

 return (
  <div>
   <h1>Counter App</h1>
   <button onClick={handleClick}>Increment</button>
   <MyChildCount1 childNo={1} count={count} />
  </div>
 );
}

function MyChildCount1(props) {
 return (
  <div>
   <h2>Child No: {props.childNo}</h2>
   <p>Count: {props.count}</p>
  </div>
 );
}
```

7. Introducing multiple child components to get an idea about the prop drilling

```
function App() {
 const [count, setCount] = useState(0);
 const handleClick = () => {
  setCount(count + 1);
 };

 return (
  <div>
   <h1>Counter App</h1>
   <button onClick={handleClick}>Increment</button>
   <MyChildCountCountainer childenCount={4} count={count} />
  </div>
 );
```

```
}

function MyChildCount1(props) {
  return (
    <div>
      <section>
        <h2>Child No: {props.childNo}</h2>
        <p>Count: {props.count}</p>
      </section>
    </div>
  );
}

function MyChildCountCountainer(props) {
  const myRange = Array.from({ length: props.childenCount }, (_, i) => i + 1);
  return (
    <div>
      {myRange.map((id) => (
        <MyChildCount1 childNo={id} count={props.count} />
      ))}
    </div>
  );
}
```

8. Introducing reducers to handle the events

```
const initialState = { count: 0 };

function reducer(state, action) {
  switch (action.type) {
    case "increment":
      return { count: state.count + 1 };
    case "decrement":
      return { count: state.count - 1 };
    case "refresh":
      return { count: 0 };
    default:
      throw new Error();
  }
}

function App() {
  const [state, dispatch] = useReducer(reducer, initialState);

  const handleClick = () => {
    dispatch({ type: "increment" });
  };
```

```
  const handleDecrement = () => {
    dispatch({ type: "decrement" });
  };

  const handleRefresh = () => {
    dispatch({ type: "refresh" });
  };

  return (
    <div>
      <h1>Count: {state.count}</h1>
      <button onClick={handleClick}>Increment</button>
      <button onClick={handleDecrement}>Decrement</button>
      <button onClick={handleRefresh}>Refresh</button>
    </div>
  );
}
```

Using Redux

1. Create a new react project and remove all the unnecessary files and codes.
2. Install redux store.

npm install redux react-redux

3. Install @reduxjs/toolkit

npm install @reduxjs/toolkit

4. Create a file named “store.js” and add the following code

```
import { configureStore } from "@reduxjs/toolkit";
import counterReducer from "./CounterSlice";

export default configureStore({
   reducer: {
      counter: counterReducer
   },
  });
```

5. Create a file named “CounterSlice.js” and add the following code.

```
import { createSlice } from "@reduxjs/toolkit";
```

```
export const counterSlice = createSlice({
   name: 'counter',
   initialState: {
     value: 0,
   },
   reducers: {
     increment: (state) => {
       // Redux Toolkit allows us to write "mutating" logic in reducers. It
       // doesn't actually mutate the state because it uses the Immer library,
       // which detects changes to a "draft state" and produces a brand new
       // immutable state based off those changes
       state.value += 1
     },
     decrement: (state) => {
       state.value -= 1
     },


     incrementByAmount: (state, action) => {
       state.value += action.payload
     },
   },
 })

 // Action creators are generated for each case reducer function
 export const { increment, decrement, incrementByAmount } = counterSlice.actions

 export default counterSlice.reducer
```

6. Create a new file named “Counter.jsx” and add the following.

```
import React from "react";
import { useSelector, useDispatch } from "react-redux";
import { decrement, increment } from "./CounterSlice";

export default function Counter() {
 const count = useSelector((state) => state.counter.value);
 const dispatch = useDispatch();

 return (
  <div>
   <div>
    <button
     aria-label="Increment value"
     onClick={() => dispatch(increment())}
    >
```

```
        +
      </button>
      <span>{count}</span>
      <button
        aria-label="Decrement value"
        onClick={() => dispatch(decrement())}
      >
        -
      </button>
    </div>
  </div>
 );
}
```

7. Modify the App.js as follows.

```
import { useState } from "react";
import "./App.css";
import Counter from "./Counter";

function App() {
  return (
    <>
      <Counter />
    </>
  );
}

export default App;
```

8. Finally, modify the main.jsx as follows.

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App.jsx";
import "./index.css";
import { Provider } from "react-redux";
import store from "./store.js";

ReactDOM.createRoot(document.getElementById("root")).render(
```

```
  <React.StrictMode>
    <Provider store={store}>
      <App />
    </Provider>
  </React.StrictMode>
);
```